

# Document Intelligence Assistant

Detailed 4-Phase Implementation Plan & Technology Stack

**Ephemeral, session-free RAG agent for user-uploaded documents**

Evaluated on CUAD (Contract Understanding Atticus Dataset)

## Design Summary

This plan builds an agent that ingests a user-uploaded PDF into a temporary, in-memory index at query time, answers questions against it with a self-correcting agentic loop, cites the exact source page, and discards the index when the session ends. No persistent database or per-user vector storage is used. The pipeline is validated during development using CUAD's 510 annotated contracts as a repeatable, ground-truth test corpus.

### Framework decision: LangChain / LlamaIndex vs. hand-rolled

| Component                   | Recommended approach                                                   | Reasoning                                                                                                                                                              |
|-----------------------------|------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Document loading & chunking | LangChain — RecursiveCharacterTextSplitter + ParentDocumentRetriever   | Matches the parent-child chunking pattern directly; less code to maintain than a hand-rolled splitter, at the cost of slightly less visibility into exact split points |
| Embeddings                  | LangChain — HuggingFaceEmbeddings wrapper around sentence-transformers | Same underlying model as before; the wrapper just standardizes the interface so the embedding backend can be swapped later without touching calling code               |
| Vector index                | LangChain-native FAISS / Chroma vectorstore classes                    | Same libraries underneath (faiss-cpu, chromadb); LangChain's VectorStore interface is used so it plugs directly into ParentDocumentRetriever and EnsembleRetriever     |
| Hybrid fusion (RRF)         | LangChain — EnsembleRetriever (BM25Retriever + vector retriever)       | Implements weighted rank fusion out of the box; read its source before your viva so you can explain the fusion math precisely rather than saying “the library does it” |
| Agentic control loop        | LangGraph                                                              | The state-graph model matches the retrieve/grade/generate/check loop exactly and is the current standard for this pattern                                              |
| LLM calls                   | Direct Anthropic/OpenAI SDK (no LangChain LLM wrapper)                 | Keeps prompt construction and output parsing fully visible and debuggable                                                                                              |
| Evaluation                  | Ragas                                                                  | Purpose-built for RAG evaluation; not worth reimplementing faithfulness/recall metrics by hand                                                                         |

*Updated recommendation: use LangChain for ingestion, chunking, embeddings, and hybrid retrieval (Phases 1–2), since these map closely onto existing LangChain components (ParentDocumentRetriever, EnsembleRetriever) and reduce code you'd otherwise maintain by hand. Keep LLM calls on the direct SDK and keep LangGraph for the agent loop in Phase 3 — that boundary hasn't changed. Table extraction (camelot-py) still needs custom glue code either way, since LangChain's splitters don't natively understand “do not split this chunk.”*

### Complete technology stack

| Layer       | Technology             | Model / library version | Role                               |
|-------------|------------------------|-------------------------|------------------------------------|
| PDF parsing | pdfplumber, camelot-py | pdfplumber ≥ 0.10       | Extracts text with layout; camelot |

| Layer                                | Technology                                                          | Model / library version                                           | Role                                                                                  |
|--------------------------------------|---------------------------------------------------------------------|-------------------------------------------------------------------|---------------------------------------------------------------------------------------|
|                                      |                                                                     |                                                                   | extracts tables to structured form                                                    |
| Chunking                             | LangChain — RecursiveCharacterTextSplitter, ParentDocumentRetriever | langchain-text-splitters $\geq 0.2$                               | Parent-child hierarchical chunking, page-number metadata carried in document metadata |
| Embedding model                      | LangChain HuggingFaceEmbeddings (sentence-transformers backend)     | all-MiniLM-L6-v2 (fast) or BAAI/bge-small-en-v1.5 (more accurate) | Local, free, CPU-friendly dense embeddings behind a swappable interface               |
| Keyword search                       | LangChain BM25Retriever (rank_bm25 backend)                         | BM25Okapi                                                         | Sparse lexical search for exact terms, clause numbers, figures                        |
| Fusion                               | LangChain EnsembleRetriever                                         | Weighted rank fusion (RRF-based)                                  | Combines dense + sparse retrievers into one ordered list                              |
| Vector index                         | LangChain FAISS / Chroma vectorstore                                | faiss-cpu, or chromadb.EphemeralClient()                          | In-memory only — built per upload, discarded at session end                           |
| Agent orchestration                  | LangGraph                                                           | langgraph $\geq 0.2$                                              | Retrieve → Grade → (rewrite loop) → Generate → Hallucination-check state graph        |
| LLM (generation + grading)           | Claude or GPT API                                                   | Claude Sonnet or GPT-4o-mini class model                          | No GPU required; used for grading, generation, and hallucination checks               |
| Evaluation framework                 | Ragas                                                               | ragas $\geq 0.1$                                                  | Faithfulness, context precision/recall, answer relevancy                              |
| Backend API                          | FastAPI                                                             | fastapi + uvicorn                                                 | Handles upload endpoint, builds ephemeral index, streams chat responses               |
| Frontend                             | Streamlit                                                           | streamlit $\geq 1.30$                                             | Upload widget, chat interface, citation display panel                                 |
| Reranker (optional, Phase 3 stretch) | sentence-transformers CrossEncoder                                  | cross-encoder/ms-marco-MiniLM-L-6-v2                              | Re-orders top-20 fused results before final top-5 selection                           |

## PHASE 1 — Ingestion & Ephemeral Indexing Pipeline

Weeks 1–3

### Objective

Given any uploaded PDF, produce a fully searchable, in-memory hybrid index within seconds — no persistence, no external database.

### Detailed tasks

1. Build a PDF loader using pdfplumber that extracts text per page while preserving reading order for multi-column layouts.
2. Add camelot-py table detection; convert detected tables to clean Markdown and keep each table as a single, unsplit LangChain Document.
3. Wrap page text as LangChain Documents (with page\_num, doc\_id in metadata), then chunk using RecursiveCharacterTextSplitter for child chunks (200–400 tokens) and a coarser splitter for parent chunks.
4. Wire both splitters into a ParentDocumentRetriever so small chunks are searched but full parent context is returned on retrieval.
5. Attach metadata to every document: doc\_id (per-upload UUID), page\_num, chunk\_type (text/table), parent\_id, token\_count.
6. Embed child chunks using LangChain's HuggingFaceEmbeddings (all-MiniLM-L6-v2) and load into an in-memory FAISS or Chroma vectorstore via ParentDocumentRetriever's built-in indexing.
7. Build a LangChain BM25Retriever over the same chunk text, in memory, alongside the vector store.
8. Wrap the full pipeline in a single function: build\_index(pdf\_bytes) -> retriever, with no disk writes.

### Models / frameworks used in this phase

- pdfplumber, camelot-py — parsing (no ML model, rule-based extraction)
- LangChain RecursiveCharacterTextSplitter + ParentDocumentRetriever — parent-child chunking
- LangChain HuggingFaceEmbeddings / all-MiniLM-L6-v2 — embedding model, runs on CPU, ~80MB
- LangChain FAISS / Chroma vectorstore — in-memory only, wrapped by ParentDocumentRetriever
- LangChain BM25Retriever (rank\_bm25 backend) — lexical index
- LangGraph is not used in this phase — introduced in Phase 3 for the agent loop only

### Testing approach

Test against 5–10 CUAD contracts first, since their layouts are realistic (multi-column clauses, embedded tables, signature blocks) and any parsing failures will surface quickly. Manually inspect a sample of retrieved parent/child chunks to confirm tables were not split and page metadata survived the LangChain Document pipeline before moving to Phase 2.

### Deliverable

**A single reusable function that takes raw PDF bytes and returns a ready-to-query, in-memory hybrid index in under ~10 seconds for a typical 20–30 page contract.**

## PHASE 2 — Hybrid Retrieval & Fusion

Weeks 4–5

### Objective

Given a natural-language question and the ephemeral index from Phase 1, return the most relevant chunks by combining meaning-based and keyword-based search.

### Detailed tasks

9. Wrap the Phase 1 vectorstore as a LangChain retriever (via `ParentDocumentRetriever` or `vectorstore.as_retriever()`) returning top-20 by dense similarity.
10. Instantiate a LangChain `BM25Retriever` over the same chunk text, returning top-20 by keyword score.
11. Combine both into a LangChain `EnsembleRetriever` with configurable weights (e.g. 0.5/0.5 to start); this performs the fused ranking internally so no manual RRF loop is needed.
12. Read `EnsembleRetriever`'s source / docs closely and note its exact fusion behavior so it can be explained precisely, not just referenced, in a viva.
13. (Stretch) Add a cross-encoder reranker (ms-marco-MiniLM-L-6-v2) as a second pass on the top-20 ensemble results, producing a final top-5.
14. Return each retrieved document with its metadata (page\_num, chunk\_type) attached, ready for citation in Phase 3.
15. Build a small retrieval-quality test harness: given a CUAD question and its ground-truth clause, check whether the correct chunk appears in your top-5 results (hit rate @ 5).

### Models / frameworks used in this phase

- Same embedding model as Phase 1 (all-MiniLM-L6-v2, via LangChain `HuggingFaceEmbeddings`) — must match for query/document vector compatibility
- LangChain `BM25Retriever` — same BM25 index built in Phase 1
- LangChain `EnsembleRetriever` — weighted fusion of the dense and BM25 retrievers
- cross-encoder/ms-marco-MiniLM-L-6-v2 — optional reranker, sentence-transformers `CrossEncoder` class

### What to measure

| Metric                                   | What it tells you                                                                               |
|------------------------------------------|-------------------------------------------------------------------------------------------------|
| Hit rate @ 5 (dense only)                | Baseline retrieval quality using embeddings alone                                               |
| Hit rate @ 5 (BM25 only)                 | Baseline retrieval quality using keywords alone                                                 |
| Hit rate @ 5 (EnsembleRetriever, hybrid) | Should be measurably higher than either alone — this comparison is a key result for your report |

### Deliverable

A `retrieve(query, index)` function returning the fused, cited top-k chunks, plus a small results table showing hybrid search beats either method alone on a CUAD sample.

## PHASE 3 — Agentic Reasoning Loop

Weeks 6–8

### Objective

Wrap retrieval in a self-correcting agent loop so the system checks its own context and its own answer before responding — this is the centerpiece of the project.

### Detailed tasks

16. Set up a LangGraph StateGraph with a shared state object holding: query, retrieved\_chunks, rewrite\_count, answer, is\_grounded.
17. Retriever node: calls the Phase 2 retrieve() function.
18. Grader node: prompts the LLM with the query + retrieved chunks, asking it to judge (yes/no + reason) whether the context is sufficient to answer.
19. Conditional edge: if the grader says no and rewrite\_count < 2, route to a Query Rewriter node (LLM rewrites the question, e.g. expanding abbreviations or rephrasing), then back to the Retriever. Otherwise proceed.
20. Generator node: prompts the LLM to answer strictly from the retrieved chunks, and to output the answer with explicit page-number citations in a structured (JSON) format.
21. Hallucination-check node: prompts the LLM to verify each factual claim in the generated answer is directly supported by the retrieved text; if unsupported claims are found, either strip them or return a lower-confidence response.
22. Add a hard cap: after 2 failed rewrite attempts, return a best-effort answer with an explicit “low confidence” caveat rather than looping indefinitely.

### Models / frameworks used in this phase

- LangGraph — the one framework dependency in the whole project, used specifically for the state graph
- Claude or GPT API — used identically across all four LLM-calling nodes (grader, rewriter, generator, hallucination-checker); only the prompt differs per node
- No fine-tuning in this phase — all four node behaviors are achieved through prompting, not model training

### Example prompt skeletons

- Grader: “Given the question and these retrieved passages, answer only YES or NO: can this question be fully answered using only this context?”
- Generator: “Answer the question using only the provided context. For every claim, cite the page number in brackets. If the context is insufficient, say so explicitly.”
- Hallucination-checker: “For each sentence in this answer, confirm whether it is directly supported by the provided context. List any unsupported sentences.”

### Deliverable

**An answer(pdf\_bytes, question) function that runs the full loop end-to-end and returns a grounded, cited answer or an explicit refusal.**

## PHASE 4 — Evaluation, Interface & Demo Polish

Weeks 9–12

### Objective

Prove the system works with numbers, wrap it in a usable interface, and prepare a clean, convincing demo.

### Detailed tasks

23. Build a Streamlit app: file upload widget → calls `build_index()` from Phase 1 → chat panel calls `answer()` from Phase 3 → citation panel shows retrieved chunk text and page number.
24. Assemble a held-out evaluation set of 40–50 questions from CUAD's annotated clause categories, with their ground-truth answer spans.
25. Run the full pipeline against every evaluation question; log results into Ragas and compute: Faithfulness, Context Precision, Context Recall, Answer Relevancy.
26. Add guardrails: reject non-PDF uploads, handle empty or scanned (image-only) PDFs gracefully with a clear error, and test out-of-domain questions to confirm the system refuses rather than hallucinates.
27. Stress-test the ephemeral index lifecycle: confirm memory is released and no chunk data persists after a session ends.
28. Record the demo video: upload a contract the system has never seen → ask a real question live → show the cited answer — in one continuous take.
29. Write the final report: architecture, design decisions (including why LangChain was adopted for Phases 1–2 while LLM calls and the agent loop stay on the direct SDK/LangGraph, and why the design is ephemeral rather than persistent), and the full evaluation results table.

### Models / frameworks used in this phase

- Ragas — evaluation only, wraps calls to your existing pipeline and the same LLM API
- FastAPI — optional, only needed if Streamlit's built-in server isn't sufficient for your deployment target
- Streamlit — interface layer, no ML model of its own

### Final evaluation targets

| Metric                                      | Target                                                                        |
|---------------------------------------------|-------------------------------------------------------------------------------|
| Faithfulness (Ragas)                        | > 0.85 — answers should almost always be fully supported by retrieved context |
| Context recall @ 5                          | > 0.75 — the correct clause should usually be in the top-5 retrieved chunks   |
| Refusal accuracy on out-of-domain questions | > 0.90 — the system should reliably decline rather than guess                 |

### Deliverable

A working Streamlit demo, a documented evaluation results table, a demo video, and a final written report.

## Summary: what makes this defensible

- LangChain is used precisely where its components fit the task (parent-child chunking, hybrid retrieval fusion) — not as a catch-all wrapper around everything, so the pieces you'd be asked to explain (fusion weighting, chunk boundaries) stay traceable to a specific, named component.
- The one framework used (LangGraph) is used for exactly the part it's built for — a self-correcting control loop — not as a catch-all wrapper.
- The ephemeral, no-database design removes an entire category of complexity (multi-tenant storage, cleanup jobs, isolation bugs) without weakening the core RAG capability.
- CUAD supplies rigorous, lawyer-verified evaluation data without requiring you to hand-label a test set yourself.